

อัลกอริทึมย้อนรอย Backtracking algorithm

อ. ลือพล พิพานเมฆาภรณ์

ปัญหาการเรียงสับเปลี่ยน (Permutation) และปัญหาการจัด (Combinatorics)

- ปัญหาการเรียงสับเปลี่ยน (Permutation) เป็นปัญหาการจัดเรียง (order) สมาชิก ให้เป็นไปตามเงื่อนไข
- ปัญหาการจัด (Combinatorics) เป็นปัญหาที่เกี่ยวข้องกับการนับ (counting) และการเลือก (selection) สมาชิก โดยไม่คำนึงถึงลำดับ
- โดยทั่วไปปัญหาเหล่านี้มักมาพร้อมกับเงื่อนไขบังคับ (constraint) ของปัญหา
- มักต้องการคำตอบที่ไม่ขัดแย้งกับเงื่อนไขบังคับ

ตัวอย่างปัญหาการเรียงสับเปลี่ยน

- มีเก้าอี้ 3 ตัววางเรียงติดกัน มีเด็ก 3 คน เป็นเด็กชาย 1 คนและเด็กหญิง 2 คน จงหาจำนวนวิธีทั้งหมดในการจัดเด็กทั้งหมดนั่งเก้าอี้ โดยไม่อนุญาตให้เด็กผู้หญิงสองคนนั่งติดกัน



ตัวอย่างปัญหาการจัด

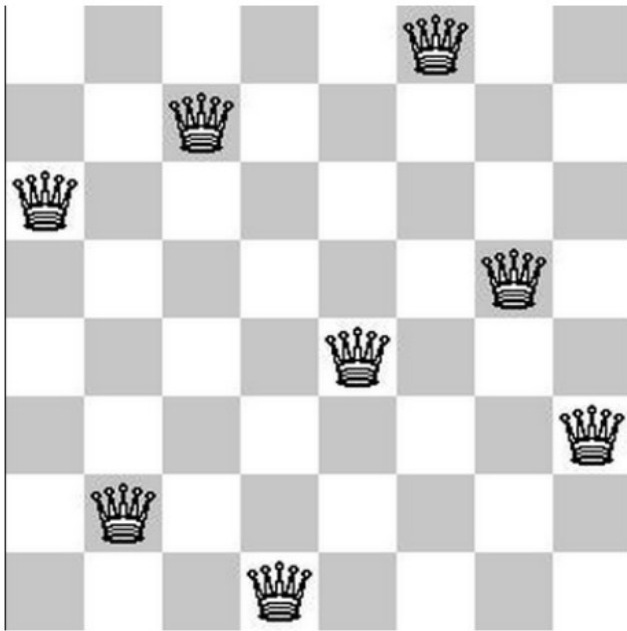
ผลรวมของเซตย่อย (Subset Sum)

- กำหนดให้ $A = \{25, 10, 9, 2, 1\}$ จงหาเซตย่อยทั้งหมดของ A ที่มีผลรวมของสมาชิกเท่ากับ 12
 - $\{10, 2\}$ และ $\{9, 2, 1\}$
- หรืออาจจะถามว่าเซตย่อยที่เล็กที่สุดที่มีผลรวมเท่ากับ 12 คือเซตใด
 - กรณีนี้คำตอบจะมีเพียง $\{10, 2\}$ เท่านั้น

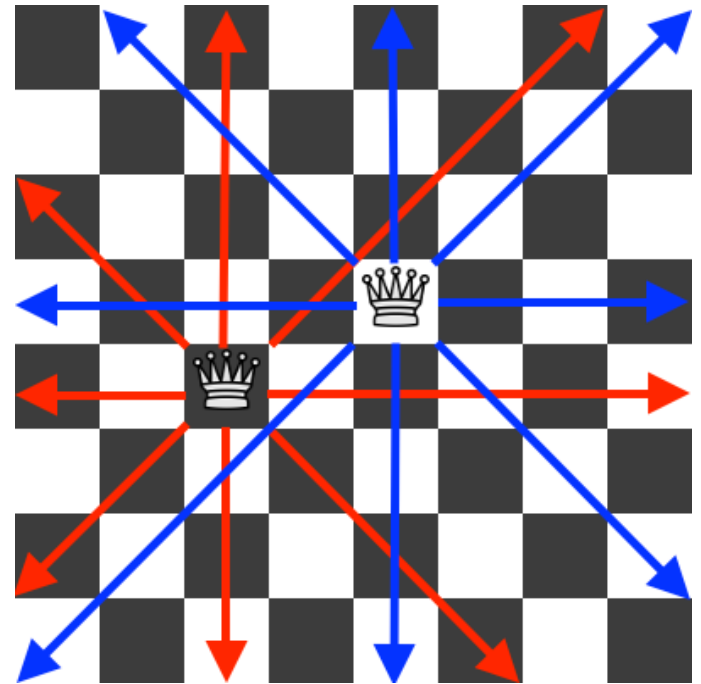
ปัญหา N ควีน

- จงหาจำนวนวิธีทั้งหมดที่สามารถวางควีน (queen) 8 ตัว ลงบนบอร์ดขนาด 8 แถว 8 หลัก โดยไม่มีควีนใดที่วางอยู่ในแนวเดียวกันในทุกทิศทาง

ทิศทาง



8 ควีน



Sudoku เกมสับสน

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

กระดานเกมสับสนเริ่มต้น

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

กระดานคำตอบ

ปัญหาถุงเป้ (Knapsack problem)

- ต้องการเลือกไอเทม (item) n ชิ้น ซึ่งมีน้ำหนัก (weight) และมูลค่า (profit) แตกต่างกัน ใส่ลงในเป้ (knapsack) ซึ่งรับน้ำหนักได้ W กิโลกรัม โดยให้มูลค่ารวมของไอเทมในเป้มีค่ามากที่สุด และถุงไม่ขาด



Wt. = 5
Value = 10



Wt. = 3
Value = 20



Wt. = 8
Value = 25



Wt. = 4
Value = 8



→ **Maximum wt. = 13**

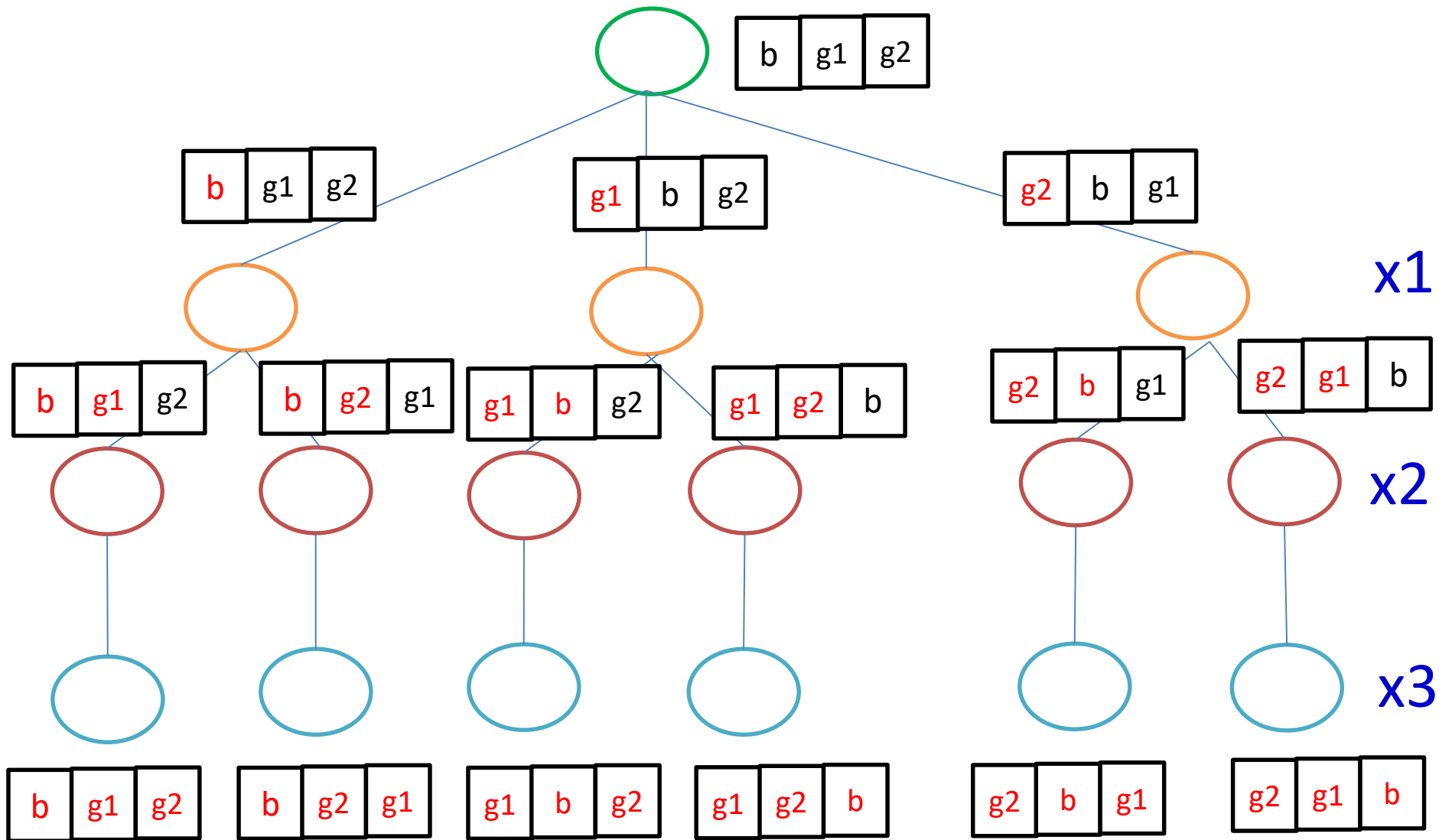
Exhaustive Search

- หลักการที่ใช้ในการค้นหาคำตอบของปัญหาเหล่านี้ ก็คือการสำรวจคำตอบที่เป็นไปได้ทั้งหมด เรียกว่า Brute force หรือ blind search จากนั้นนำคำตอบทั้งหมดที่สำรวจได้มาทดสอบกับเงื่อนไขบังคับของปัญหา เพื่อเลือกเฉพาะคำตอบที่ต้องการ
- อัลกอริทึม Exhaustive search มี 2 ขั้นตอนหลัก ได้แก่
 1. การเข้ารหัสคำตอบ
 2. วิธีสำรวจคำตอบ

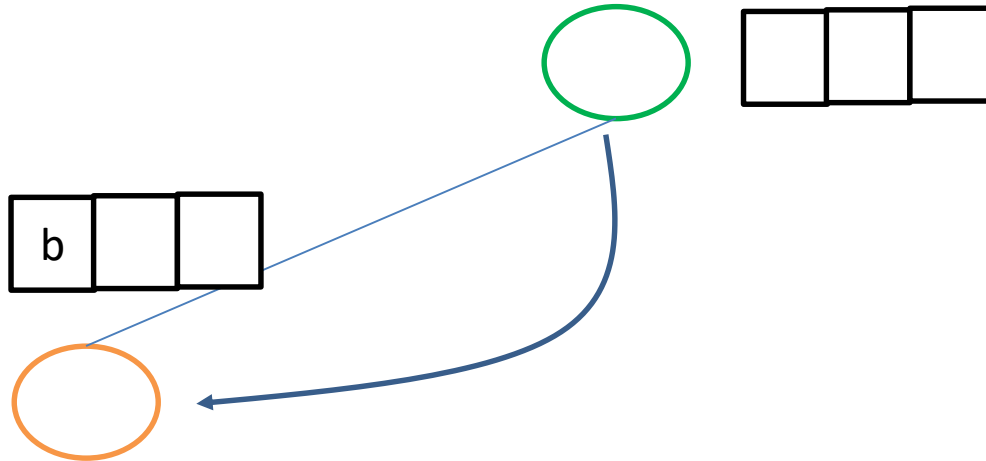
Exhaustive search (2)

- มักนิยามคำตอบในรูปของเซต (set) ตัวอย่างเช่น
 - การแสดงคำตอบของปัญหาจัดเก็้อ้อ อาจนิยามเซตคำตอบ $X = \{x_1, x_2, x_3\}$ โดยลำดับ x_1, x_2 , และ x_3 จะแทนเก็้อ้อแต่ละตัว และค่าของ x_1, x_2, x_3 แทนสถานะของเก็้อ้อ เช่น $X = \{\text{'boy'}, \text{'girl1'}, \text{'girl2'}\}$
- ขั้นตอนถัดไปคือการสำรวจสถานะของเก็้อ้อที่เป็นไปได้ใน X โดยการสร้างต้นไม้คำตอบ (solution tree) เพื่อสับเปลี่ยนลำดับในแต่ละตำแหน่งอย่างมีระบบ
- อย่างไรก็ตามการสร้างต้นไม้คำตอบที่สมบูรณ์แบบ (complete tree) ล่วงหนา อาจไม่ใช่วิธีที่เหมาะสมเนื่องจากต้นไม้คำตอบอาจมีขนาดใหญ่จนเกินไป

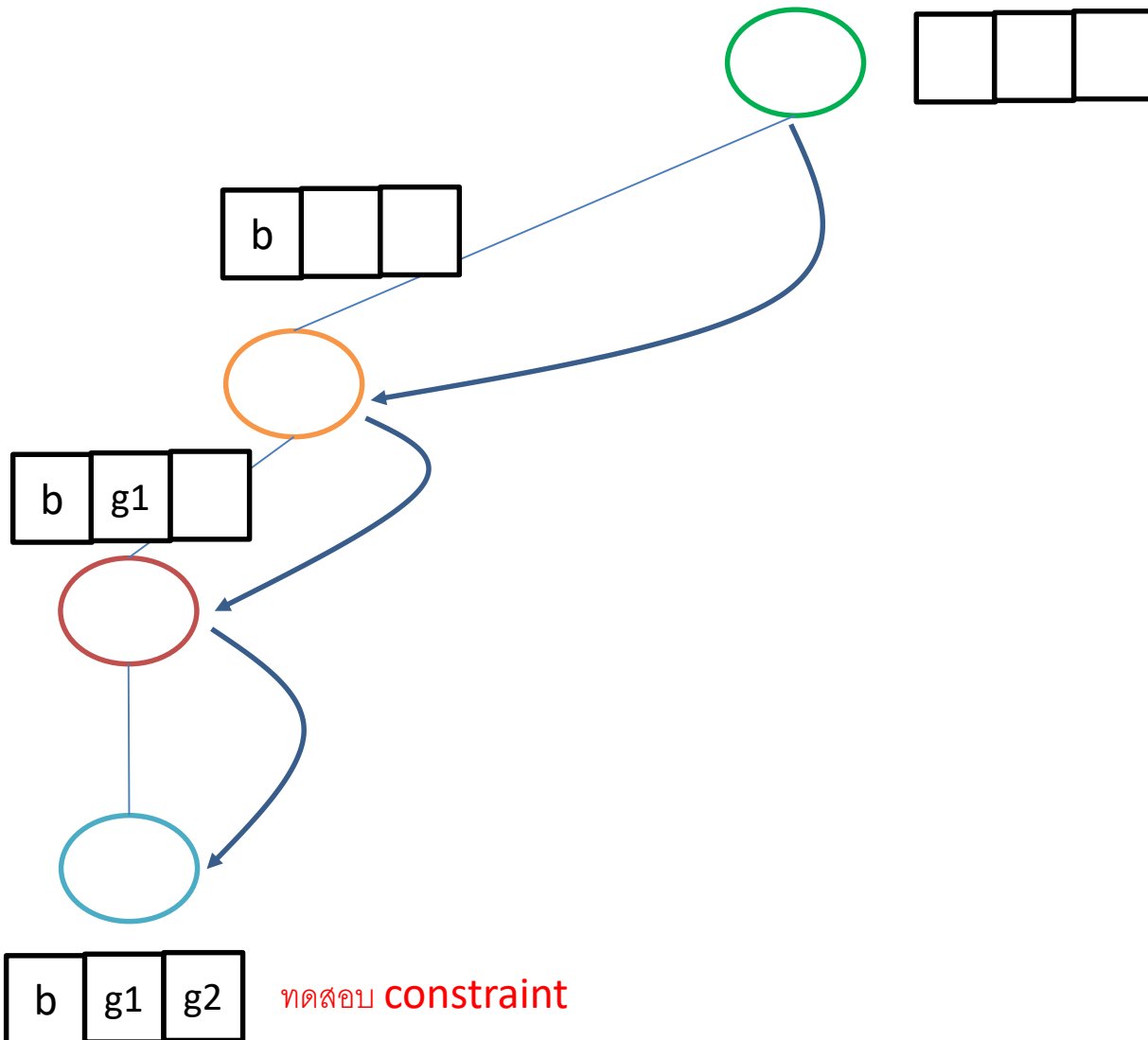
ต้นไม้คำตอบแบบสมบูรณ์ (Complete tree)



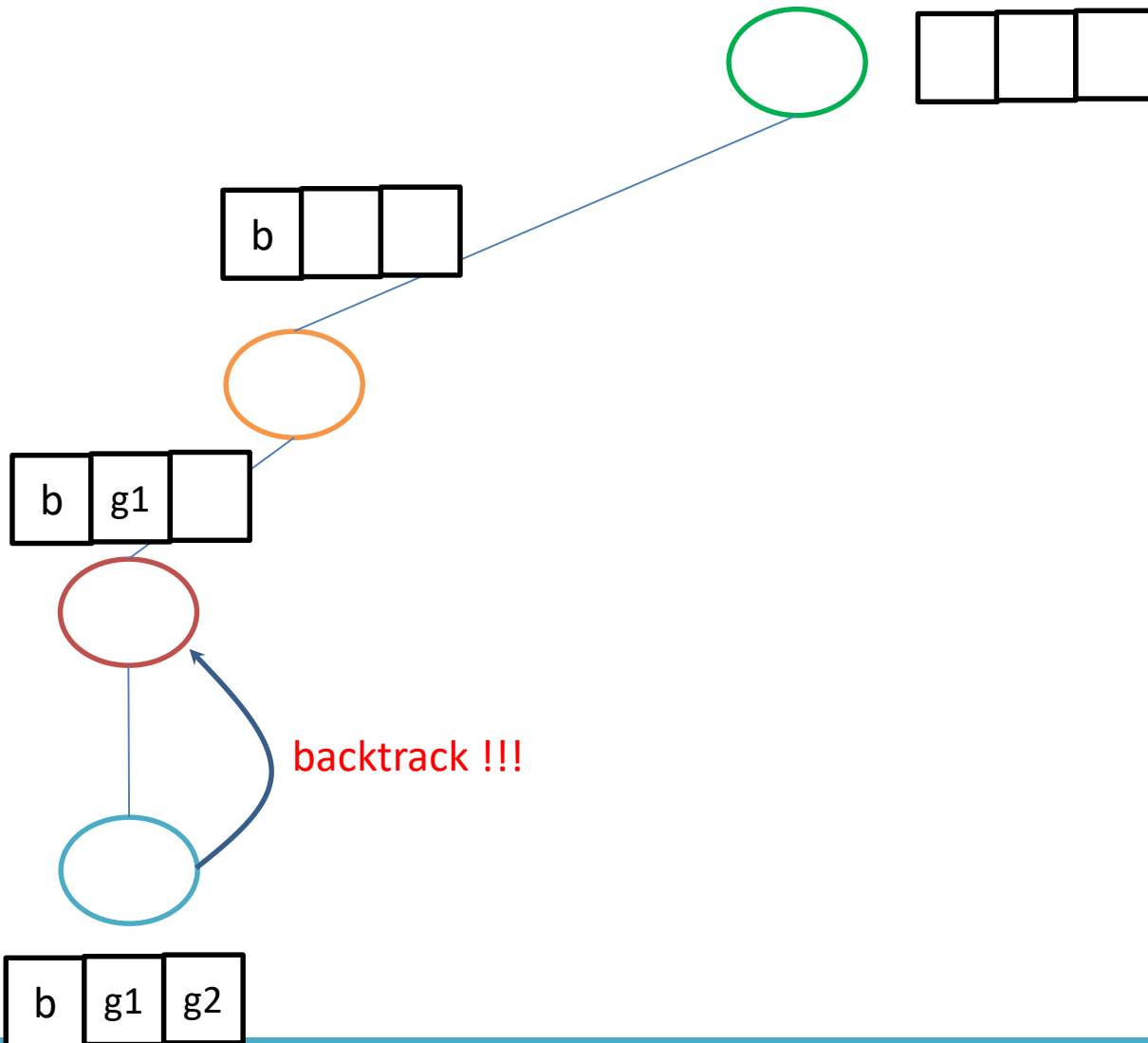
การสำรวจคำตอบด้วยวิธี Exhaustive Search



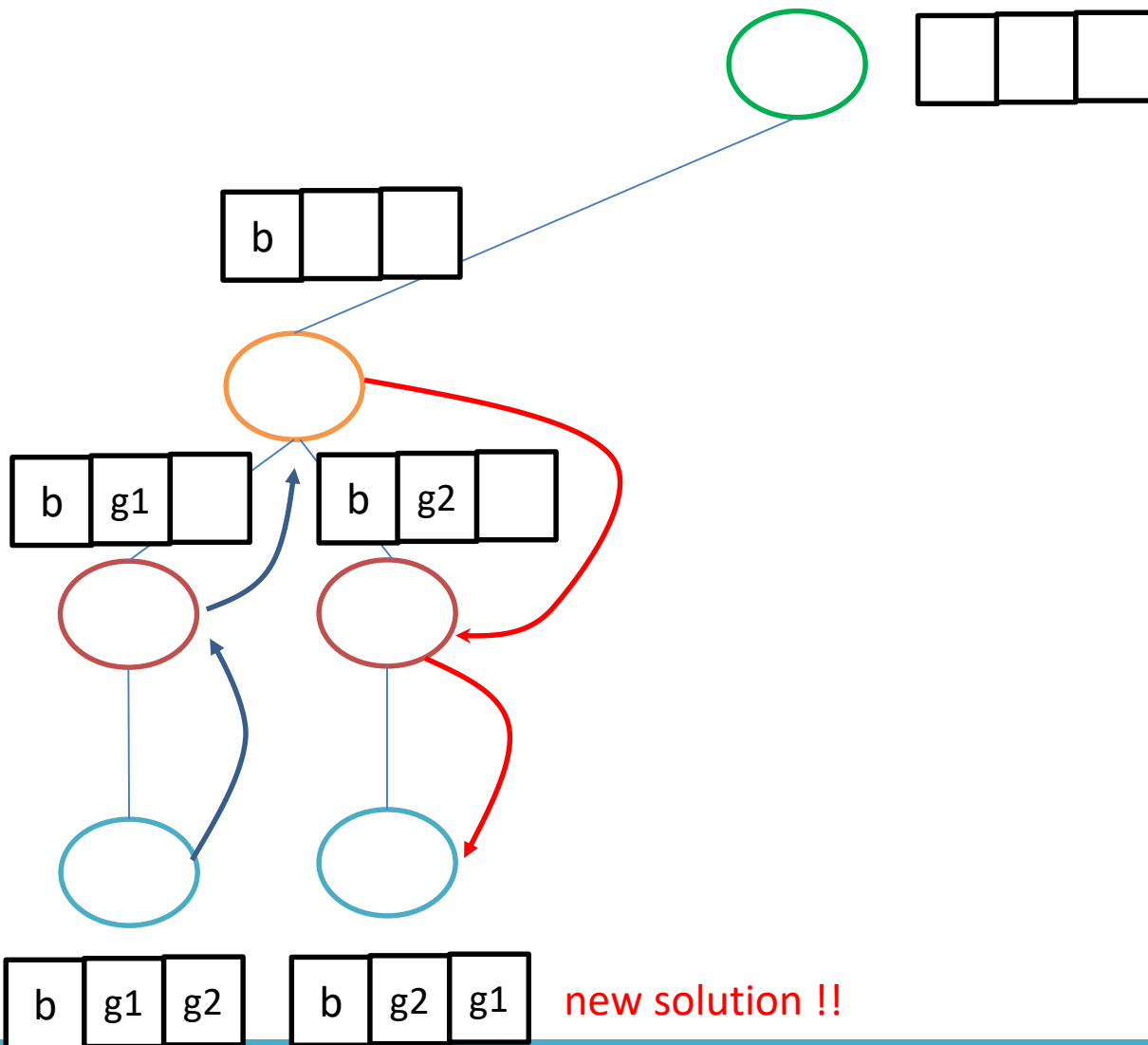
การสำรวจคำตอบด้วยวิธี Exhaustive Search



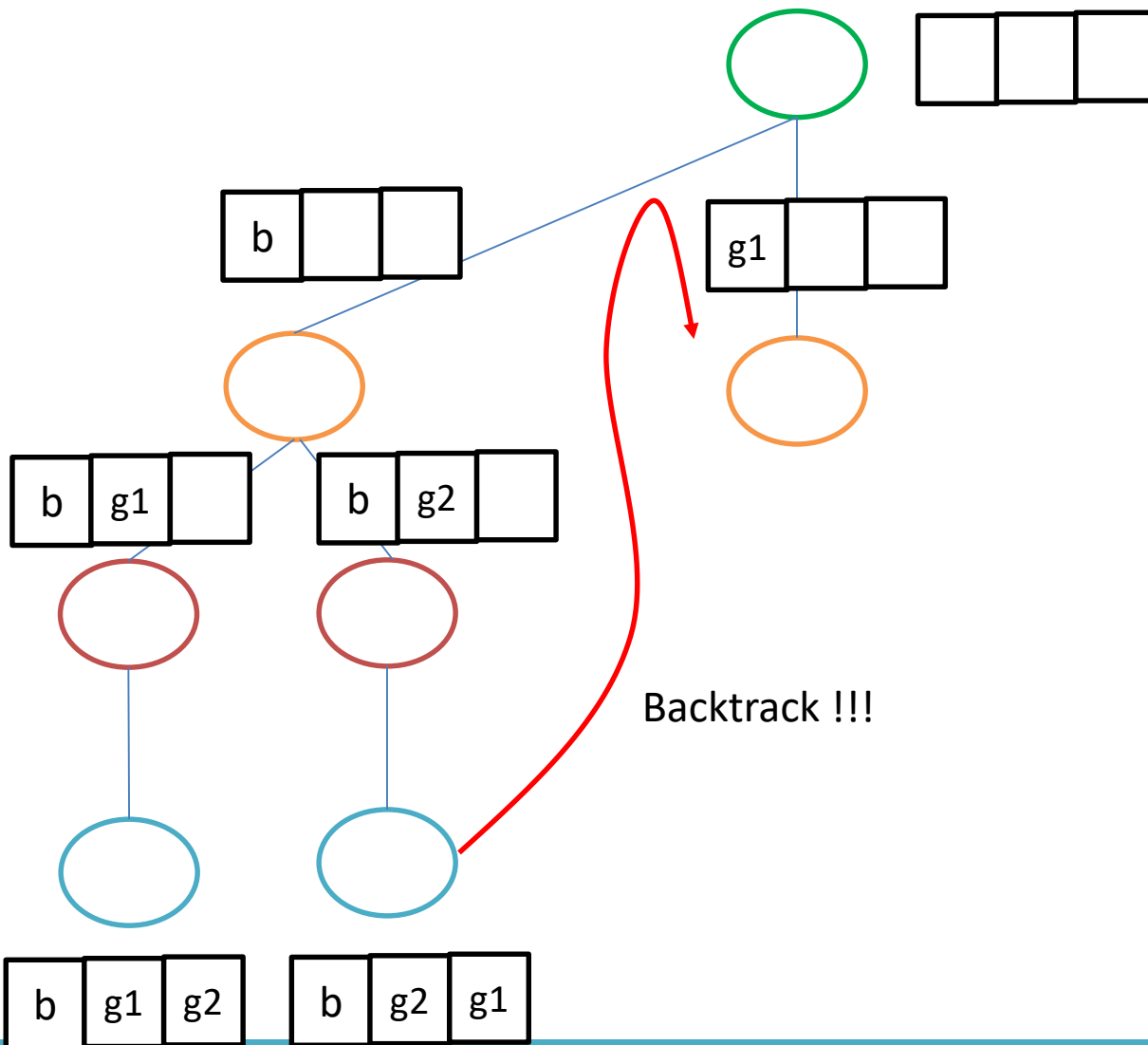
การสำรวจคำตอบด้วยวิธี Exhaustive Search



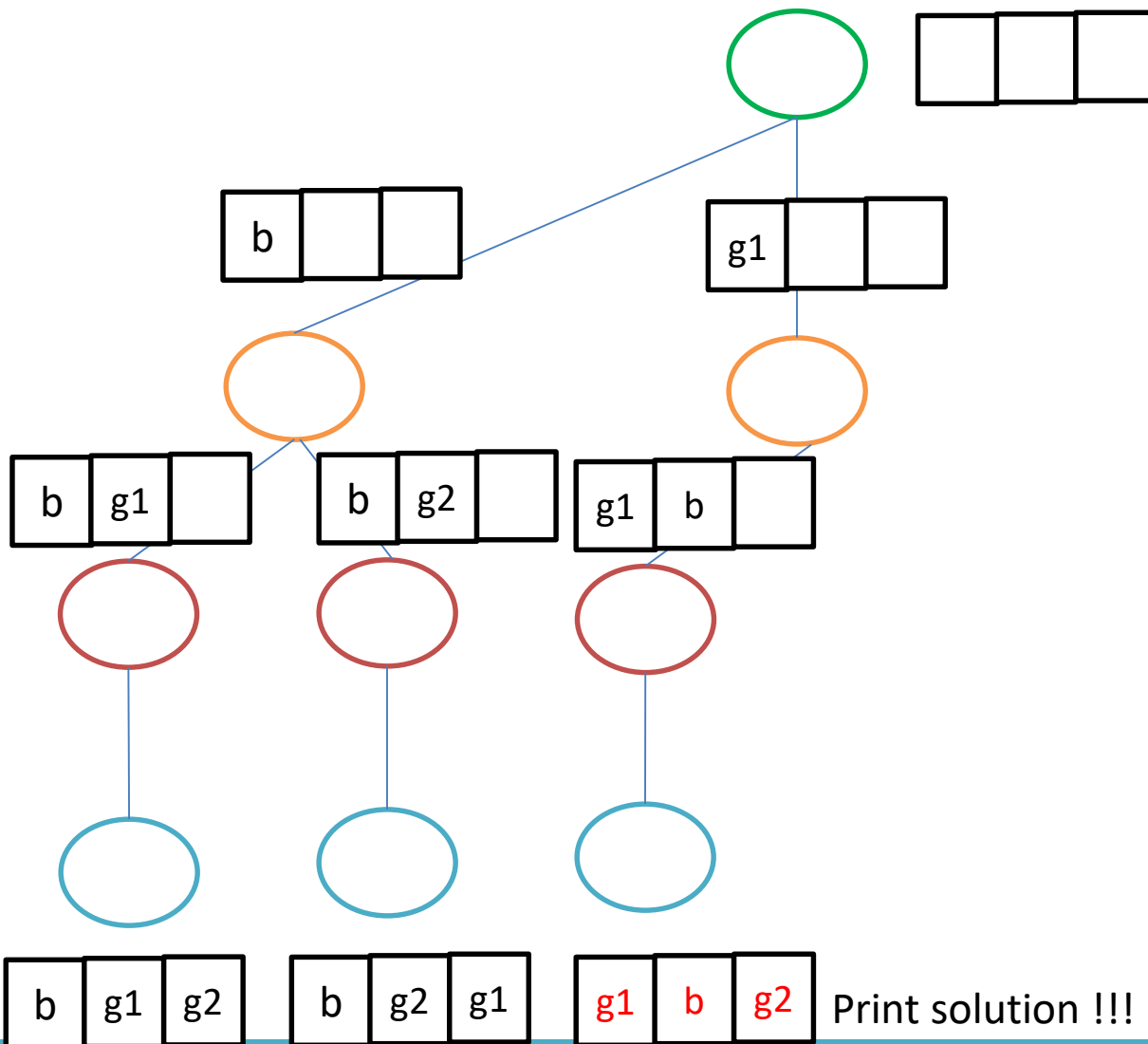
การสำรวจคำตอบด้วยวิธี Exhaustive Search



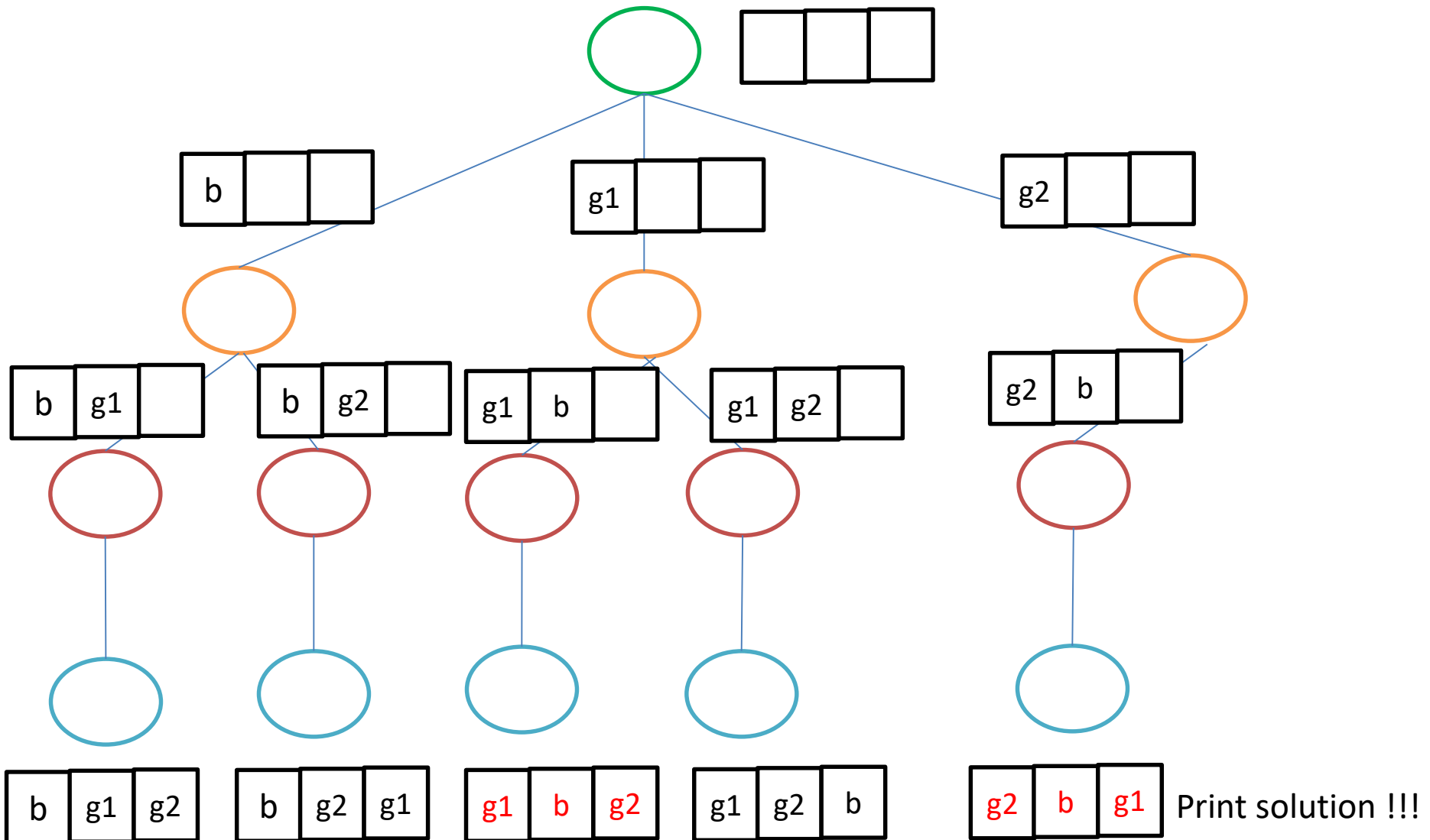
การสำรวจคำตอบด้วยวิธี Exhaustive Search



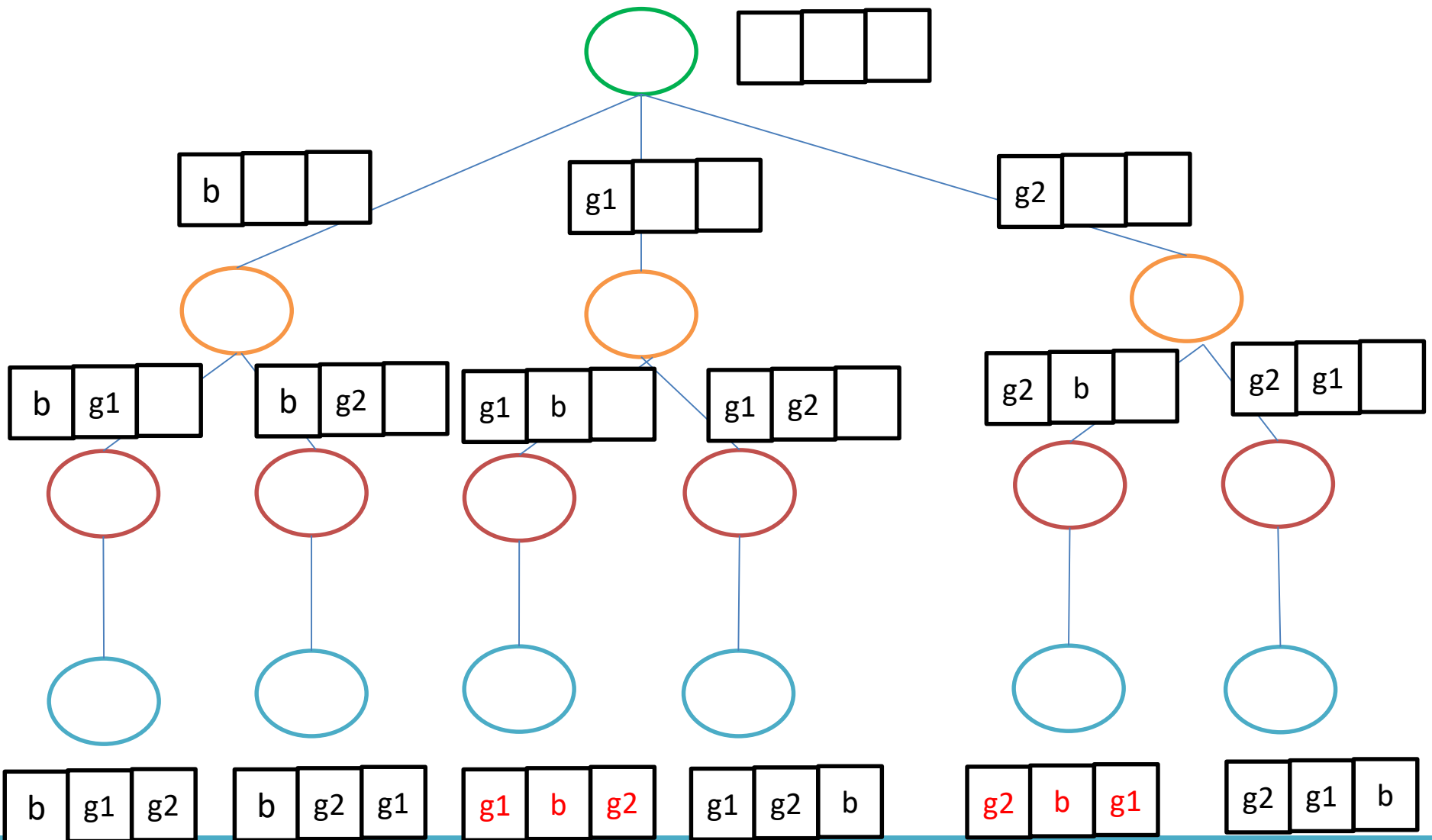
การสำรวจคำตอบด้วยวิธี Exhaustive Search



การสำรวจคำตอบด้วยวิธี Exhaustive Search



การสำรวจคำตอบด้วยวิธี Exhaustive Search



Recursive tree for Permutation

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void permute(int X[], int start, int end) {
    if (start == end) {
        // Print the current permutation
        for (int i = 1; i <= end; i++) {
            printf("%d ", X[i]);
        }
        printf("\n");
        return;
    }

    for (int i = start; i <= end; i++) {
        swap(&X[start], &X[i]); // Swap elements
        permute(X, start + 1, end); // Recursively permute the remaining elements
        swap(&X[start], &X[i]); // Undo the swap
    }
}

int main() {
    int n = 3;
    int X[] = {-1, 1, 2, 3}; // 1 = boy, 2 = g1, 3 = g2, -1 = no use

    permute(X, 1, n);

    return 0;
}
```

1 2 3
1 3 2
2 1 3
2 3 1
3 2 1
3 1 2

Workshop 1

- จงปรับปรุงโค้ด permutation ให้แสดงเฉพาะคำตอบที่ไม่ซ้ำกัน
เงื่อนไขบังคับ (เด็กผู้หญิง 2 คนต้องไม่นั่งติดกัน)

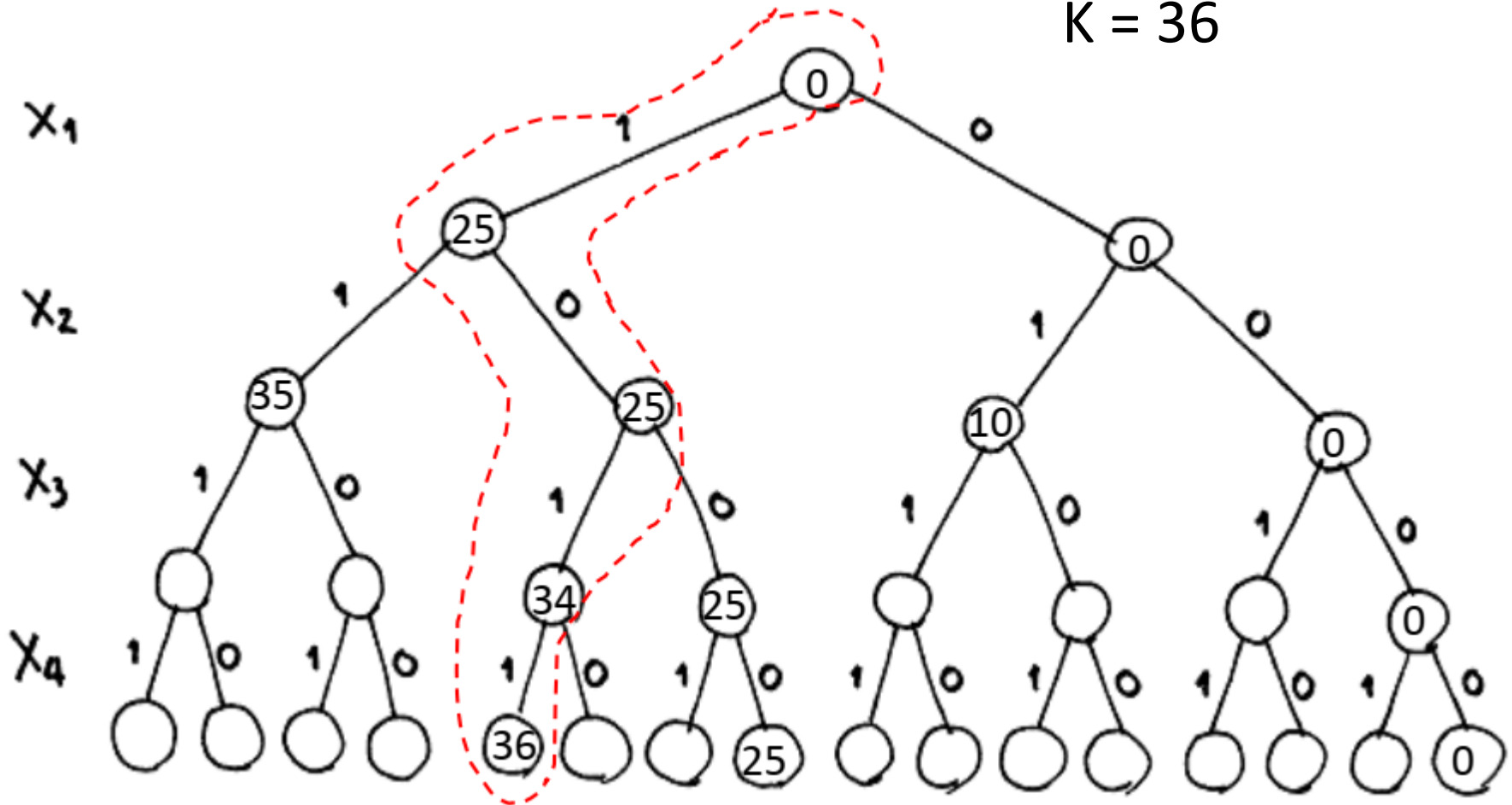


Subset Sum with Exhaustive Search

- กำหนดให้ $A = \{25, 10, 9, 2\}$ จงหาเซตย่อยที่ให้ผลรวมเท่ากับ 36
- นิยามคำตอบของปัญหานี้แทนด้วย $X = \{x_1, x_2, x_3, x_4\}$ โดยที่ x_i แทนสถานะของสมาชิกแต่ละตัวของ A และ ค่า $x_i = '1'$ หมายถึงสมาชิกตำแหน่ง i ถูกเลือก (include) และ ค่า $x_i = '0'$ หมายถึงสมาชิกในตำแหน่งที่ i ไม่ถูกเลือก (exclude)
- ตัวอย่างเช่น $\{25, 9\}$ แทนด้วยเซตคำตอบ $X = \{1, 0, 1, 0\}$ และ $\{10, 9, 2\}$ แทนด้วยเซตคำตอบ $X = \{0, 1, 1, 1\}$ เป็นต้น

Solution tree of subset sum (แบบที่ 1)

$A = \{25, 10, 9, 2\}$
 $K = 36$



Subset1

(generate binary codes)

```
#include <stdio.h>

void print_sol(int x[], int n) {
    for (int i = 1; i <= n; i++) {
        printf("%d ", x[i]);
    }
    printf("\n");
}

void subset1(int x[], int l, int r) {
    if (l == r) {
        print_sol(x, r);
    } else {
        x[l + 1] = 1;
        subset1(x, l + 1, r);
        x[l + 1] = 0;
        subset1(x, l + 1, r);
    }
}

int main() {
    int n = 3; // Number of elements
    int x[n + 1]; // Array to store subset

    subset1(x, 0, n);
    return 0;
}
```

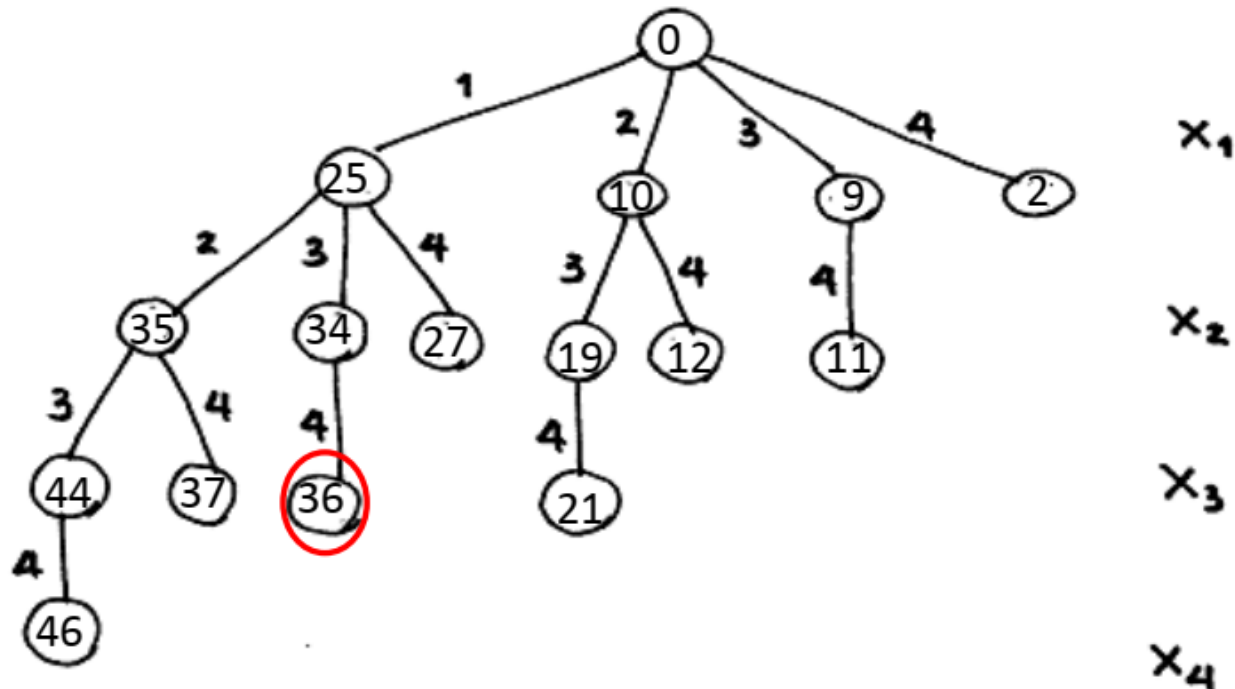
1	1	1
1	1	0
1	0	1
1	0	0
0	1	1
0	1	0
0	0	1
0	0	0

Workshop 2

- จงปรับปรุงโค้ด subset1 เพื่อแสดงค้นหาเซตย่อยใน $A = \{25, 10, 9, 2\}$ ที่มีผลรวมเท่ากับ 36

Solution tree of subset sum (แบบที่ 2)

- คำตอบของปัญหา Subset sum อาจเข้ารหัสในรูปของเซตของลำดับ index ของสมาชิกใน A เช่น $A = \{25, 10, 9, 2\}$, $k = 36$
 - $X = \{1, 3, 4\}$ แทนคำตอบ $\{25, 9, 2\} = 25 + 9 + 2 = 36$ หรือ
 - $X = \{2, 3\}$ แทนคำตอบ $\{10, 9\} = 10 + 9 = 19$



Subset 2

Sequence of index

```
#include <stdio.h>

void print_subset(int x[], int l) {
    for (int i = 1; i <= l; i++)
        printf("%d ", x[i]);
    printf("\n");
}

void subset2(int x[], int l, int n) {
    print_subset(x, l); // print current sol.
    int j;
    if(l == 0)
        j = 1; // start from the 1st element
    else
        j = x[l] + 1; // continue to next element

    for (int i=j; i<=n; i++) {
        x[l + 1] = i; // extend subsets
        subset2(x, l + 1, n); // recursive to process the next level
    }
}

int main() {
    int n = 3; // Number of elements
    int x[n+1]; // Array to store subset indices
    subset2(x, 0, n);
    return 0;
}
```

```
1
1 2
1 2 3
1 3
2
2 3
3
```

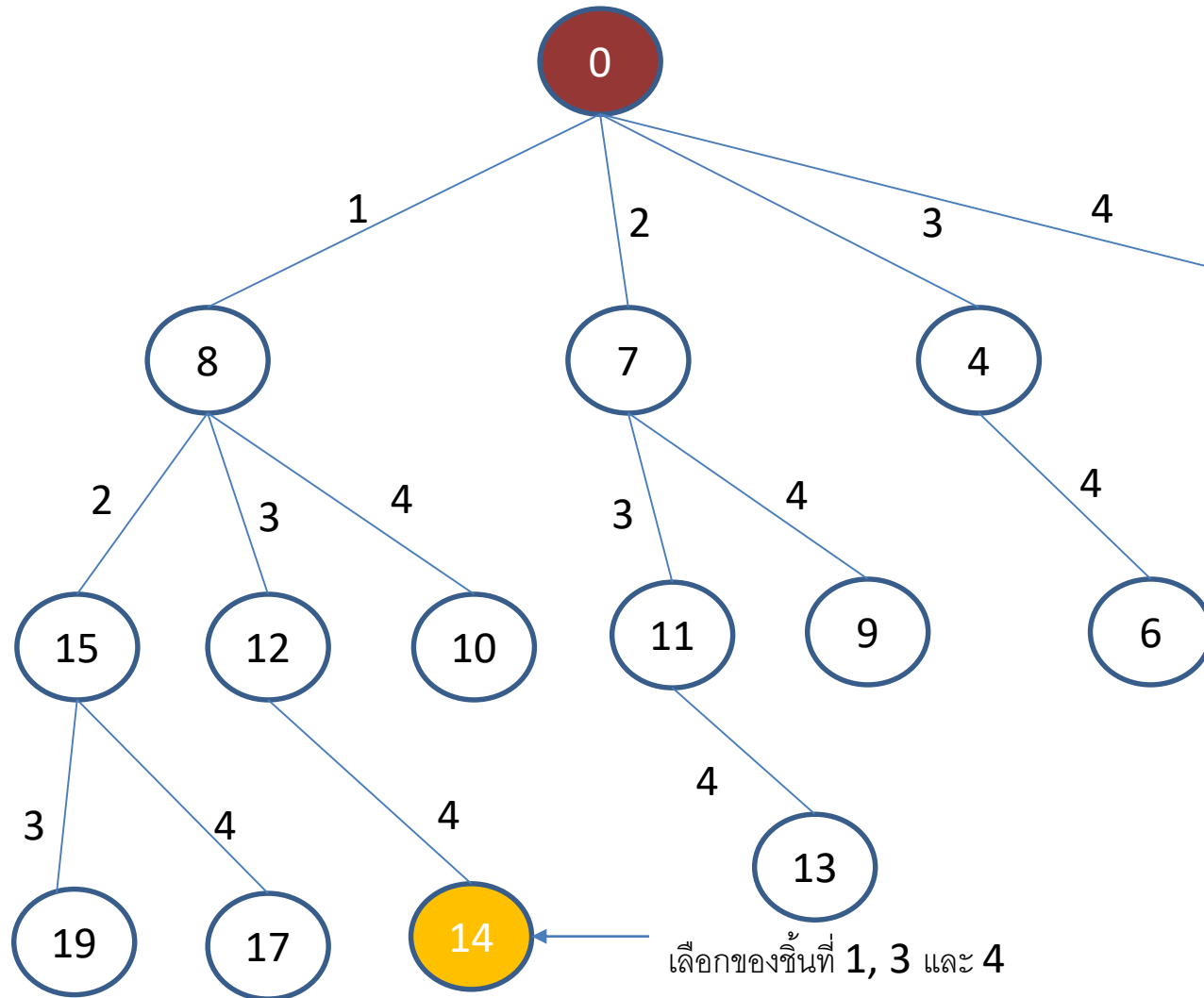
Knapsack problem

- สำหรับไบนารีเซต อาจแสดงคำตอบเป็นสถานะการเลือก/ไม่เลือกของแต่ละชิ้น $\{x_1, x_2, \dots, x_n\}$ โดยที่ $x_i \in \{0, 1\}$ เช่น $\{1, 0, 1, 0, 1, 0\}$ หมายถึงใส่ของชิ้นที่ 1, 3 และ 4 ลงไปในเป้ซึ่งมีน้ำหนักรวมคือ $10+7+6 = 23$ และมูลค่ารวมคือ $10+6+2 = 18$
- หรือแสดงคำตอบเป็นเซตย่อยโดยการเก็บลำดับของ object แต่ละชิ้นที่ถูกเลือกใส่ในเป้

Object	wi	vi
O1	12	8
O2	5	7
O3	4	4
O4	2	2
Σ	23	21

$$W = 18$$

Solution tree of Knapsack problem



$W = 18$

Object	w_i	v_i
O1	12	8
O2	5	7
O3	4	4
O4	2	2
Σ	23	21

เลือกของชิ้นที่ 1, 3 และ 4

มูลค่ารวม $8 + 4 + 2 = 14$

น้ำหนักรวม $12 + 4 + 2 = 18$

วาด Solution tree แบบ binary code สำหรับ knapsack problem

กำหนดให้สินค้า 3 ชิ้น แต่ละชิ้นมีมูลค่า $V = \{8, 4, 7\}$ และน้ำหนัก $W = \{12, 5, 4\}$
ถุงรับน้ำหนักสูงสุด 18

การเข้ารหัสคำตอบของ N queens

	0	1	2	3
0			♔	
1	♔			
2				♔
3		♔		

queen	1	2	3	4
row	1	3	0	2
col	0	1	2	3

แบบที่ 1

ต้นไม้คำตอบจะมีขนาดใหญ่มาก เนื่องจากต้องเก็บแถวและคอลัมน์
ต้องทำการ generate ค่อนข้างมาก

col	0	1	2	3
row	1	3	0	2

แบบที่ 2

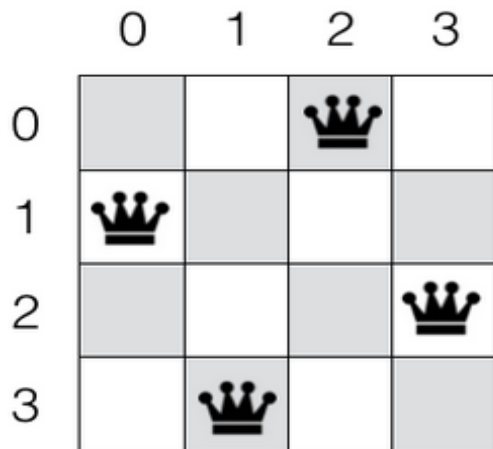
ต้นไม้คำตอบจะกระชับและใช้ space ลดลง

การตรวจสอบคำตอบใน N queens

- ทดสอบเงื่อนไขการวางตำแหน่งของควีนแต่ละตัวบนบอร์ด ดังนี้

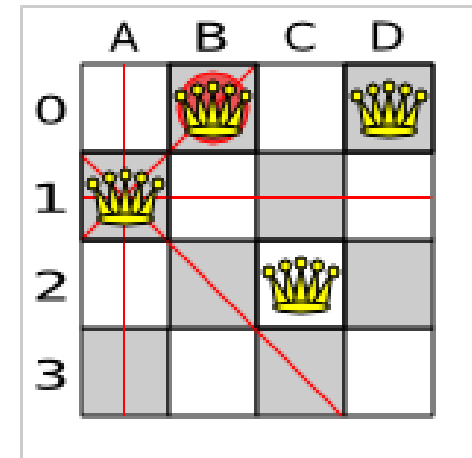
1). $|X.col - Y.col| = |X.row - Y.col|$ หรือ

2). $X.row = Y.row$



Q1: (0, 1) และ Q2: (1, 3)

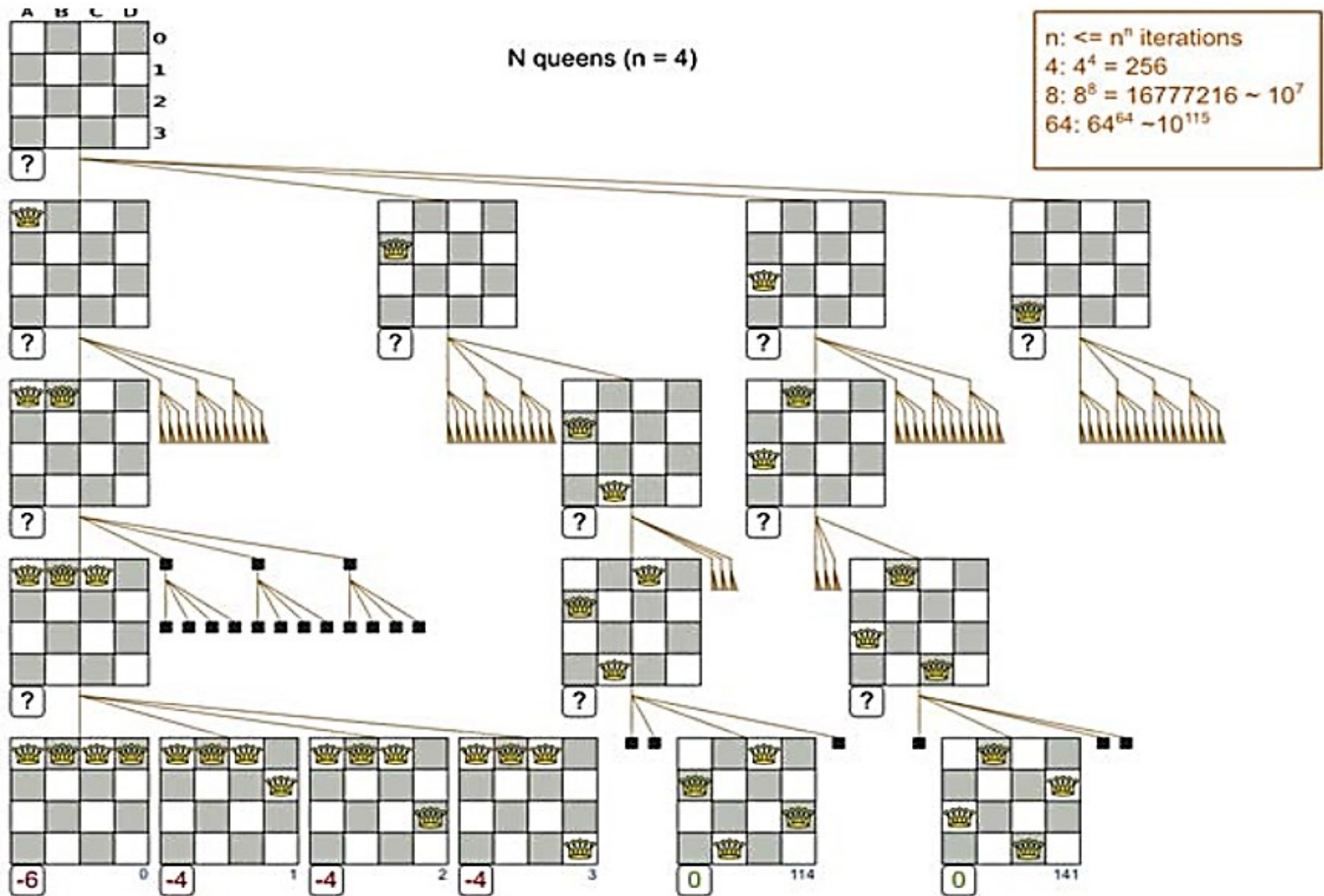
$$|0 - 1| <> |1 - 3|$$



Q1: (0, 1) และ Q2: (1, 0)

$$|0 - 1| = |1 - 0|$$

Board tree (แบบที่ 1)



4 Queens with

Exhaustive Search

```
int n=4, x[4];

void n_queens(int k) {
    if (k == n) {          // place all queens on the board
        int flag = 0;

        for(int j = 1; j < k ; j++)    // check any two queens
        {
            for(int p = 0 ; p < j ;p++)
            {
                if(abs(p-j) == abs(x[p]-x[j]) ||
                   (x[j] == x[p]))
                    flag = 1;
            }
        }

        if(flag == 0)      // no conflict print sol.
        {
            for(int i=0; i<k; i++)
                printf("%d ", x[i]);
            printf("\n");
        }
    } else {
        for (int row = 0; row < n; row++) { // place queen k on row i
            x[k] = row;
            n_queens(k + 1);    // place next queen
        }
    }
}

int main() {
    n_queens(0);
    return 0;
}
```

1	3	0	2
2	0	3	1

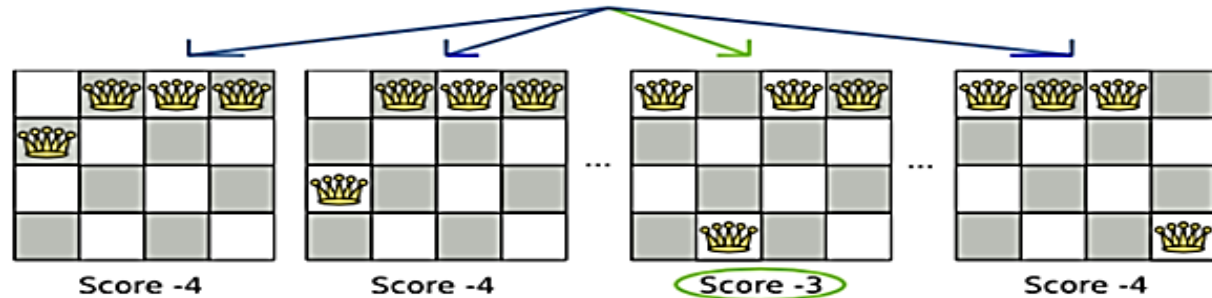
Solution tree of 4 queens ของบอร์ดแบบที่ 2

Step 0

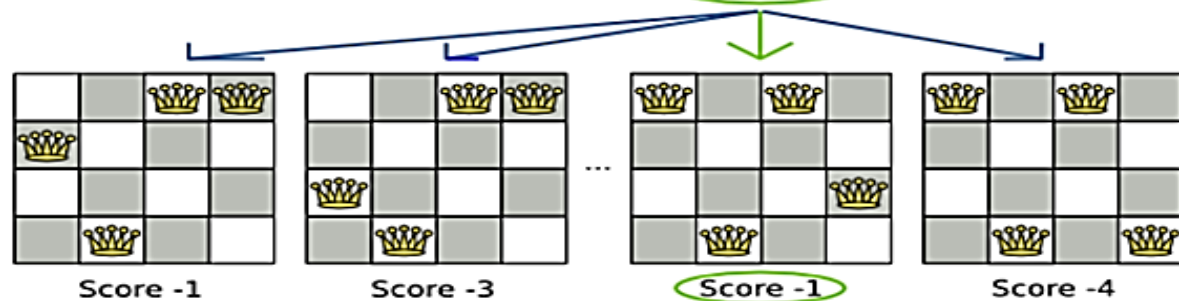
	A	B	C	D
0	♔	♔	♔	♔
1				
2				
3				

Score -6

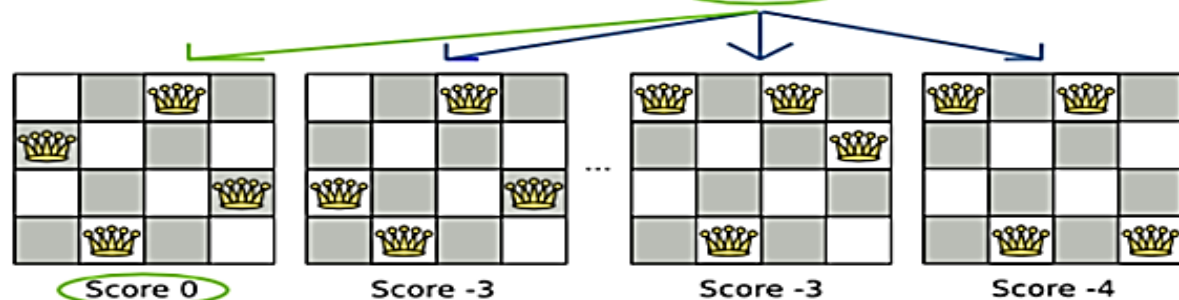
Step 1



Step 2



Step 3



ปรับปรุงโค้ด permute สำหรับ ปัญหา n-queens แบบ 2

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void permute(int X[], int start, int end) {
    if (start == end) {
        for (int i = 1; i <= end; i++) {
            printf("%d ", X[i]);
        }
        printf("\n");
        return;
    }

    for (int i = start; i <= end; i++) {
        swap(&X[start], &X[i]);
        permute(X, start + 1, end);
        swap(&X[start], &X[i]);
    }
}

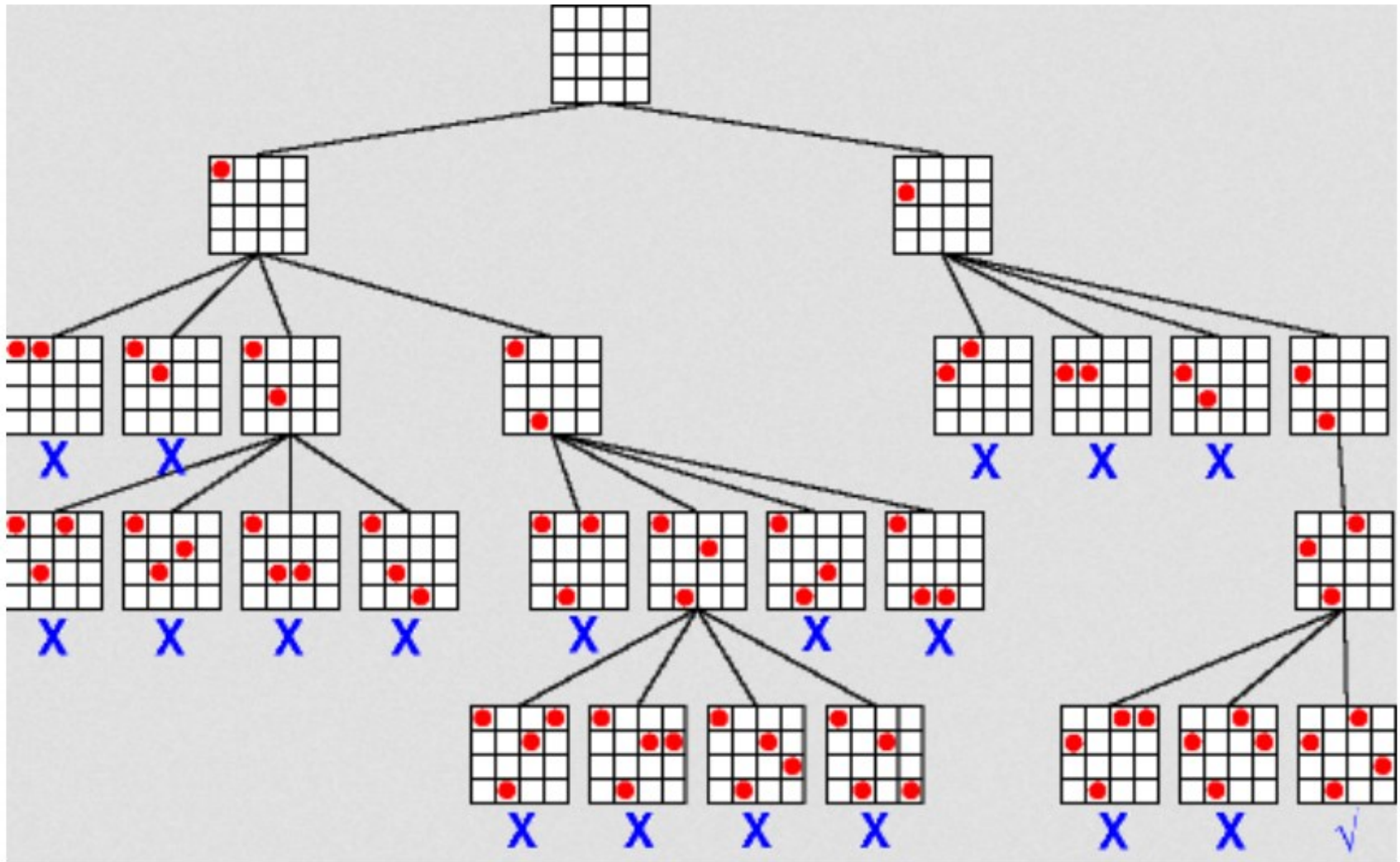
int main() {
    int n = 3;
    int X[] = {-1, 1, 2, 3};
    permute(X, 1, n);

    return 0;
}
```

อัลกอริทึมย้อนรอย (Backtracking algorithm)

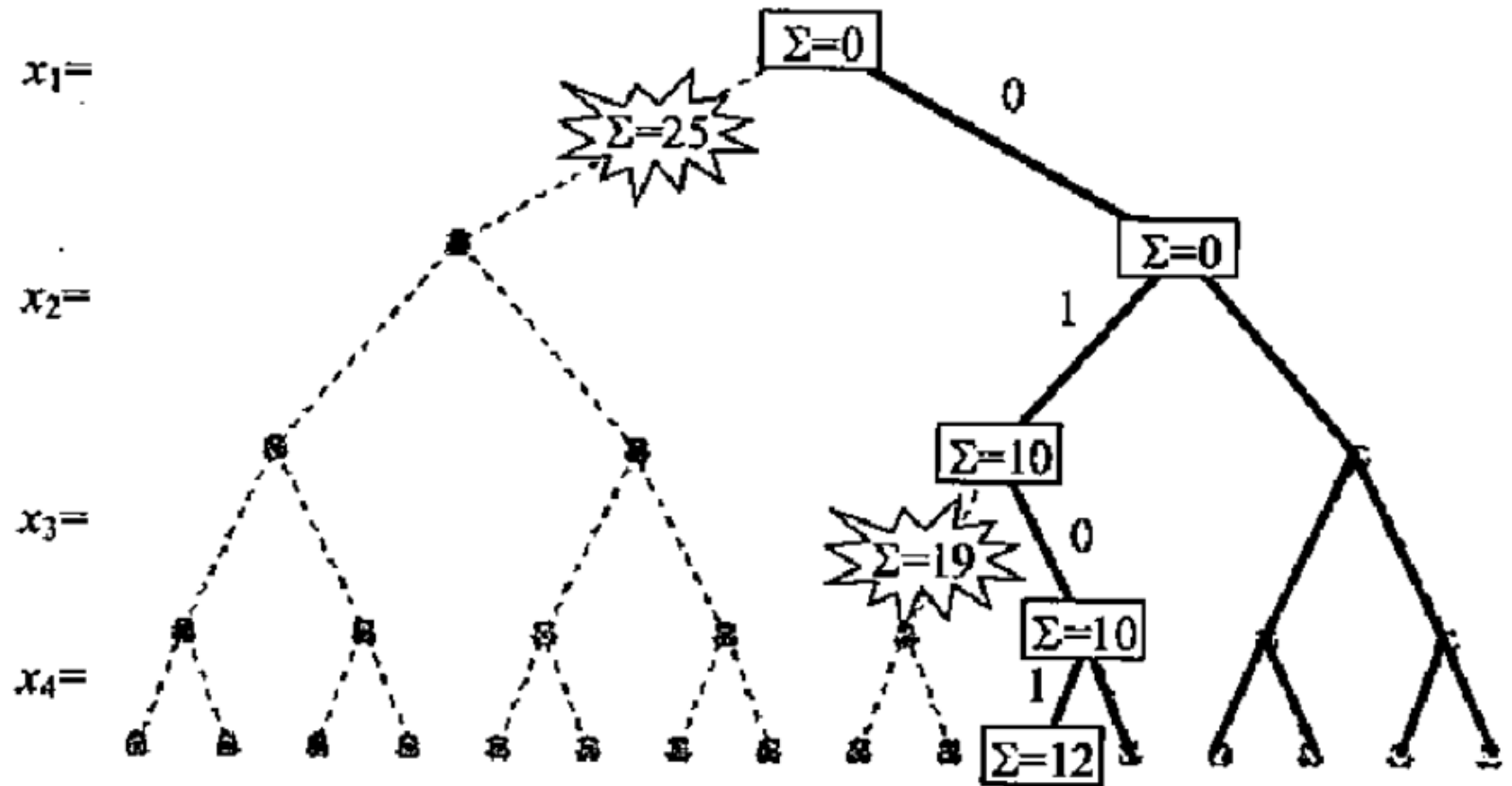
- เทคนิคช่วยเพิ่มประสิทธิภาพให้กับ Exhaustive Search เพื่อลดจำนวน search ส่งผลให้ค้นหาคำตอบได้เร็วขึ้น
- แนวคิดของอัลกอริทึมย้อนรอยคือการสำรวจสถานะคำตอบที่ละเส้นทาง (path) และตัดเส้นทาง (pruning) ที่ไม่อาจนำไปสู่คำตอบของปัญหาทิ้ง โดยการตรวจสอบสถานะของคำตอบปัจจุบันว่าขัดแย้งกับเงื่อนไขบังคับของปัญหาหรือไม่

ตัวอย่าง 4 queens with backtracking



Subset sum with backtracking

สมมติว่า $A = \{25, 10, 9, 2\}$ หาเซตย่อยที่ผลรวมเท่ากับ $P = 12$



```

#include <stdio.h>
int A[] = {25, 10, 6 ,4}; // input
k = 10;
void print_sol(int x[], int n)
{   for (int j = 1; j <= n; j++)
        if (x[j]== 1)
            printf("%d ", A[j-1]);
        printf("\n");
}

void Subset1(int x[], int l, int r, int sum)
{   if(sum == k)
        {   print_sol(x, r);
            return;
        }
        if(l==r)
            return;

        if(sum > k)
            return;

        x[l+1] = 1;
        sum += A[l];
        Subset1(x, l+1, r, sum);
        x[l+1] = 0;
        sum -= A[l];
        Subset1(x, l+1, r, sum);
}

-
int main()
{   int n = 4;
    int x[n+1];
    Subset1(x, 0, n, 0);
    return 0;
}

```

เพิ่มตัวแปร **sum** เพื่อทดสอบผลรวมของคำตอบปัจจุบันว่าเกินกว่าค่า **k** หรือไม่ ถ้าเกินก็จะทำการ **backtrack**

ปรับปรุงโค้ด n-queens โดยใช้ backtracking

```
int n=4, x[4];

void n_queens(int k) {
    if (k == n) {          // place all queens on the board
        int flag = 0;

        for(int j = 1; j < k ; j++)    // check any two queens
        { for(int p = 0 ; p < j ;p++)
            { if(abs(p-j) == abs(x[p]-x[j]) ||
              (x[j] == x[p]))
                flag = 1;
            }
        }

        if(flag == 0)      // no conflict print sol.
        { for(int i=0; i<k; i++)
            printf("%d ", x[i]);
            printf("\n");
        }
    } else {
        for (int row = 0; row < n; row++) { // place queen k on row i
            x[k] = row;
            n_queens(k + 1);    // place next queen
        }
    }
}

int main() {
    n_queens(0);
    return 0;
}
```

1	3	0	2
2	0	3	1

ปรับปรุงโค้ด subset1 ของปัญหา Knapsack โดยใช้ backtracking

```
int v[] = {12, 5, 4, 2};           // item values
int W[] = {8, 7, 4, 2};           // item weights
int max_v = -1;                   // keep max value of solution
int ans[4];                       // keep answer
int K = 18;                       // max capacity of a bag
```

```
void print_sol(int x[], int n)
{   int w_temp=0, v_temp=0;
    for (int i = 1; i <= n; i++)
    {   if (x[i] == 1)
        {   w_temp += W[i-1];
            v_temp += V[i-1];
        }
    }

    if (w_temp <= K && v_temp > max_v)
    {   max_v = v_temp;
        for (int j = 1; j <= n; j++)
            ans[j] = x[j];
    }
}
```

```
void Subset1(int x[], int l, int r)
{   if (l == r)
        print_sol(x, r);
    else
    {   x[l+1] = 1;
        Subset1(x, l+1, r);
        x[l+1] = 0;
        Subset1(x, l+1, r);
    }
}
```

W = 18

Object	wi	vi
O1	12	8
O2	5	7
O3	4	4
O4	2	2
Σ	23	21

สรุป

- อัลกอริทึม backtracking ถูกออกแบบมาเพื่อแก้ปัญหของ brute force อย่างมีประสิทธิภาพ โดยกำหนดวิธีการเข้ารหัสคำตอบ และวิธีการสำรวจคำตอบทั้งหมดที่เป็นไปได้ได้อย่างมีระบบ
- ในระหว่างการสำรวจเส้นทางของคำตอบจะมีการตัดเส้นทางของสถานะคำตอบที่ขัดแย้งกับเงื่อนไขบังคับของปัญหา เพื่อลดขนาดของ search space
- ข้อจำกัดของอัลกอริทึม backtracking
 - Exponential time complexity: หากคำตอบมีความลึกและกว้างมากๆ อาจส่งผลให้ประสิทธิภาพของอัลกอริทึมย่นรอยลดลงได้
 - Redundant or Duplicate Solutions: ความซ้ำซ้อนของคำตอบ